

CLRS Chapter 20 Study Guide

Study Guide: CLRS Chapter 20 — Elementary Graph Algorithms

Generated 2026-06-14. Subject: Computer Science (Algorithms).
Coverage: Comprehensive (no limit).

Chapter-by-Chapter Breakdown

Ch. 20 — Elementary Graph Algorithms

Named Entities (Terms & Definitions)

- **Graph $G = (V, E)$:** A data structure consisting of a set V of vertices and a set E of edges (pairs of vertices).
- **Directed graph (digraph):** Each edge (u, v) is directed from u to v .
- **Undirected graph:** Each edge (u, v) is unordered; (u, v) and (v, u) represent the same connection.
- **Adjacency-list representation:** Array Adj of $|V|$ lists; $\text{Adj}[u]$ contains all vertices v such that $(u, v) \in E$. Memory: $\Theta(V + E)$.
- **Adjacency-matrix representation:** $|V| \times |V|$ matrix A where $a_{ij} = 1$ if $(i, j) \in E$. Memory: $\Theta(V^2)$.
- **Weighted graph:** Each edge has an associated weight $w: E \rightarrow \mathbb{R}$.
- **Weight function w :** $E \rightarrow \mathbb{R}$, maps edges to real-valued weights.
- **Sparse graph:** One where $|E|$ is much less than $|V|^2$.
- **Dense graph:** One where $|E|$ is close to $|V|^2$.
- **Universal sink:** A vertex with in-degree $|V| - 1$ and out-degree 0.
- **Transpose of a directed graph G^T :** Graph with edges reversed: $E^T = \{(v, u) : (u, v) \in E\}$.
- **Incidence matrix:** $|V| \times |E|$ matrix B where $b_{ij} = \text{indicator of edge } j \text{ incident on vertex } i$.
- **Square of a directed graph G^2 :** Graph where $(u, v) \in E^2$ iff G contains a path with at most 2 edges between u and v .
- **Breadth-first search (BFS):** Graph search algorithm that explores vertices in waves radiating from a source vertex s .
- **Source vertex s :** The starting vertex for BFS.
- **Shortest-path distance $\delta(s, v)$:** Minimum number of edges in any path from s to v (∞ if unreachable).
- **Breadth-first tree:** Predecessor subgraph $G_\pi = (V_\pi, E_\pi)$ produced by BFS; contains a unique shortest path from s to every reachable vertex.
- **Predecessor subgraph G_π :** Subgraph defined by parent pointers; $V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}$ for BFS; $V_\pi = V$ for DFS.
- **Tree edges:** Edges in the depth-first forest or breadth-first tree.

- **Depth-first search (DFS):** Graph search that explores as deep as possible before backtracking.
- **Depth-first forest:** Collection of depth-first trees produced by DFS (since it searches from multiple sources).
- **Discovery time $u.d$:** Timestamp when vertex u is first discovered (turned gray) in DFS.
- **Finish time $u.f$:** Timestamp when vertex u 's adjacency list is fully explored (turned black) in DFS.
- **Parenthesis structure:** Discovery/finish times produce properly nested parentheses; if $u.d < v.d < v.f < u.f$ then v is a descendant of u .
- **White-path theorem:** v is a descendant of u in DFS iff at time $u.d$ there is a path from u to v consisting entirely of white vertices.
- **Back edge:** Edge (u, v) connecting a vertex u to an ancestor v in a DFS tree (including self-loops).
- **Forward edge:** Non-tree edge (u, v) connecting u to a proper descendant v in a DFS tree.
- **Cross edge:** Any other edge; connects vertices in same DFS tree where neither is ancestor of other, or vertices in different DFS trees.
- **Topological sort:** Linear ordering of vertices of a DAG such that if $(u, v) \in E$, u appears before v .
- **Directed acyclic graph (DAG):** A directed graph with no cycles.
- **Strongly connected component (SCC):** Maximal set of vertices $C \subseteq V$ such that for every pair $u, v \in C$, both $u \rightsquigarrow v$ and $v \rightsquigarrow u$.
- **Component graph G_SCC :** Each SCC is contracted to a single vertex; edge (C_i, C_j) exists if there is an edge from any vertex in C_i to any vertex in C_j in G . Always a DAG.
- **Semiconnected graph:** Directed graph where for all pairs (u, v) , either $u \rightsquigarrow v$ or $v \rightsquigarrow u$ (or both).
- **Singly connected graph:** Directed graph where there is at most one simple path between any two vertices.
- **Articulation point:** Vertex whose removal disconnects an undirected graph.
- **Bridge:** Edge whose removal disconnects an undirected graph.
- **Biconnected component:** Maximal set of edges in an undirected graph where any two edges lie on a common simple cycle.
- **Euler tour:** Cycle in a strongly connected directed graph that traverses each edge exactly once.
- **Diameter of a tree:** $\max\{\delta(u, v) : u, v \in V\}$, the largest shortest-path distance.
- **Planar graph:** An undirected graph that can be drawn in the plane with no edges crossing. Euler proved $|E| < 3|V|$.

Processes / Algorithms / Pathways

BFS (Breadth-First Search)

- **Type:** Algorithm (graph search)
- **Goal:** Find shortest-path distances (in number of edges) from source s to all reachable vertices; build breadth-first tree.

- **Input:** Graph $G = (V, E)$ (adjacency lists), source vertex s .
- **Output:** $v.d = \delta(s, v)$ for all $v \in V$; $v.\pi$ = predecessor in breadth-first tree.
- **Data structures:** FIFO queue Q ; each vertex has color (WHITE/GRAY/BLACK), distance d , predecessor π .
- **Steps:**
 1. Initialize: For each vertex $u \in G.V - \{s\}$, set $u.color = \text{WHITE}$, $u.d = \infty$, $u.\pi = \text{NIL}$
 2. Set $s.color = \text{GRAY}$, $s.d = 0$, $s.\pi = \text{NIL}$
 3. Create empty queue Q , $\text{ENQUEUE}(Q, s)$
 4. While $Q \neq \emptyset$:
 1. $u = \text{DEQUEUE}(Q)$
 2. For each vertex v in $G.\text{Adj}[u]$:
 3. If $v.color == \text{WHITE}$: 8. $v.color = \text{GRAY}$ 9. $v.d = u.d + 1$
 4. $v.\pi = u$
 5. $\text{ENQUEUE}(Q, v)$
 5. $u.color = \text{BLACK}$
- **Invariant:** At the test in the while loop, Q consists of all gray vertices.
- **Complexity:** $O(V + E)$ time, $\Theta(V)$ queue operations, each adjacency list scanned once.
- **Edge Cases:** Vertices not reachable from s stay WHITE with $d = \infty$. Graph can be directed or undirected.
- **Termination early:** BFS can stop early if all $|V|$ vertices have been discovered (finite d), not just when Q empties.
- **Example:** On the undirected graph with vertices s (source), a , b , c where $s-a$, $s-b$, $a-c$, $b-c$:
 - Init: $s.d = 0$, others ∞ . $Q = [s]$
 - Dequeue s , scan neighbors a , b : set $a.d = 1$, $b.d = 1$, $a.\pi = s$, $b.\pi = s$. $Q = [a, b]$
 - Dequeue a , scan neighbors s (BLACK), c (WHITE): set $c.d = 2$, $c.\pi = a$. $Q = [b, c]$
 - Dequeue b , scan neighbors s (BLACK), c (GRAY): no change. $Q = [c]$
 - Dequeue c , scan neighbors a (BLACK), b (BLACK): no change. $Q = []$. Done.
 - Distances: $s.d=0$, $a.d=1$, $b.d=1$, $c.d=2$

DFS (Depth-First Search)

- **Type:** Algorithm (graph search)

- **Goal:** Explore entire graph, computing discovery/finish times and building depth-first forest. Used as subroutine in many graph algorithms.
- **Input:** Graph $G = (V, E)$ (adjacency lists).
- **Output:** $u.d$ (discovery time), $u.f$ (finish time) for each vertex; $u.\pi$ (predecessor in DFS forest).
- **Data structures:** Global time counter; each vertex has color (WHITE/GRAY/BLACK), timestamps d and f , predecessor π .
- **Steps (DFS):**
 1. For each vertex $u \in G.V$: set $u.color = \text{WHITE}$, $u.\pi = \text{NIL}$
 2. $time = 0$
 3. For each vertex $u \in G.V$:
 4. If $u.color == \text{WHITE}$: 5. DFS-VISIT(G, u)
- **Steps (DFS-VISIT(G, u)):**
 1. $time = time + 1$
 2. $u.d = time$
 3. $u.color = \text{GRAY}$
 4. For each vertex v in $G.Adj[u]$:
 5. If $v.color == \text{WHITE}$: 6. $v.\pi = u$ 7. DFS-VISIT(G, v)
 6. $time = time + 1$
 7. $u.f = time$
 8. $u.color = \text{BLACK}$
- **Key property:** $u.d < v.d < v.f < u.f$ iff v is a proper descendant of u (Corollary 20.8).
- **Complexity:** $\Theta(V + E)$ time. DFS-VISIT called exactly once per vertex; total work scanning adjacency lists = $\Theta(E)$.
- **Edge Cases:** DFS may produce a forest (multiple trees) when graph is disconnected. Timestamps range from 1 to $2|V|$.
- **Edge classification during DFS** (by color of v when (u, v) first explored):
 - WHITE $\rightarrow$ tree edge
 - GRAY $\rightarrow$ back edge
 - BLACK $\rightarrow$ forward edge (if $u.d < v.d$) or cross edge (if $u.d > v.d$)

PRINT-PATH

- **Type:** Algorithm
- **Goal:** Print vertices on a shortest path from s to v , assuming BFS has already computed the breadth-first tree.
- **Steps:**
 1. If $v == s$: print s

2. Else if $v.\pi == \text{NIL}$: print “no path from s to v exists”
 3. Else: PRINT-PATH($G, s, v.\pi$); print v
- **Complexity:** $O(\text{length of path})$

TOPOLOGICAL-SORT

- **Type:** Algorithm
- **Goal:** Produce a linear ordering of vertices in a DAG such that all edges go forward in the ordering.
- **Input:** Directed acyclic graph $G = (V, E)$.
- **Output:** Linked list of vertices in topologically sorted order.
- **Steps:**
 1. Call DFS(G) to compute finish times $v.f$ for each vertex v
 2. As each vertex is finished, insert it onto the front of a linked list
 3. Return the linked list
- **Complexity:** $\Theta(V + E)$ — DFS dominates, insertion at front of list is $O(1)$ per vertex.
- **Correctness proof:** For any edge (u, v) , $v.f < u.f$ because (u, v) cannot be a back edge (graph is acyclic, Lemma 20.11). When v is white, it becomes a descendant of u ($v.f < u.f$). When v is black, it finished earlier ($v.f < u.f$). Thus ordering by decreasing finish time is a valid topological order.
- **Edge Cases:** If graph has a cycle, the algorithm still produces an ordering, but it will not be a valid topological sort. Lemma 20.11: G has a cycle iff DFS on G produces a back edge.

STRONGLY-CONNECTED-COMPONENTS (Kosaraju’s algorithm)

- **Type:** Algorithm
- **Goal:** Decompose a directed graph into its strongly connected components.
- **Input:** Directed graph $G = (V, E)$.
- **Output:** Each vertex labeled with its SCC membership (each DFS tree in step 3 is one SCC).
- **Steps:**
 1. Call DFS(G) to compute finish times $u.f$ for each vertex u
 2. Compute G^T (the transpose of G)
 3. Call DFS(G^T), but in the main loop, consider vertices in order of decreasing $u.f$ (from step 1)
 4. Output each tree in the depth-first forest of step 3 as a separate SCC
- **Complexity:** $\Theta(V + E)$ — two DFS passes + transpose construction.
- **Key insight:** The component graph G_{SCC} is always a DAG. DFS on G^T in decreasing finish time order from the first DFS visits SCCs in reverse topological order.
- **Edge Cases:** A single vertex with no edges is its own SCC. A graph with a Hamiltonian path may have only one SCC.

Kahn’s algorithm (topological sort — alternative)

- **Type:** Algorithm

- **Goal:** Topologically sort a DAG using in-degree counting (from Exercise 20.4-5).
- **Steps:**
 1. Compute in-degree for each vertex
 2. Initialize queue with all in-degree-0 vertices
 3. While queue not empty:
 4. Dequeue vertex u , output it
 5. For each neighbor v of u , decrement $\text{in-degree}(v)$; if $\text{in-degree}(v) == 0$, enqueue v
 6. If not all vertices output, graph has a cycle
- **Complexity:** $O(V + E)$
- **Edge Cases:** If graph has cycles, some vertices never reach in-degree 0 and remain in the queue $\rightarrow$ algorithm detects cycles.

Classifications & Hierarchies

Graph representations: - Adjacency-list: $\Theta(V + E)$ memory, good for sparse graphs - Adjacency-matrix: $\Theta(V^2)$ memory, good for dense graphs or when $O(1)$ edge lookup needed

Edge types in DFS: - Tree edges: in the depth-first forest - Back edges: to an ancestor (self-loops included) - Forward edges: to a proper descendant (non-tree) - Cross edges: all others (same tree, neither ancestor; or different trees)

Edge types by color during DFS: | Target color | Edge type | |—|—| |
 WHITE | Tree edge | | GRAY | Back edge | | BLACK | Forward ($u.d < v.d$) or Cross ($u.d > v.d$) |

In undirected graphs: Only tree edges and back edges exist (Theorem 20.10).

Vertex colors in BFS/DFS: - WHITE: undiscovered - GRAY: discovered, but not all neighbors explored yet (on the frontier) - BLACK: finished (all neighbors explored)

Graph connectivity types: - Connected (undirected): every vertex reachable from every other - Strongly connected (directed): every vertex reachable from every other - Semiconnected (directed): for all u, v : $u \rightsquigarrow v$ or $v \rightsquigarrow u$ - Singly connected (directed): at most one simple path between any two vertices

Comparisons & Trade-offs

Dimension	Adjacency List	Adjacency Matrix
Memory	$\Theta(V + E)$	$\Theta(V^2)$
Edge lookup	$O(\text{degree})$ worst-case	$O(1)$
List all edges	$\Theta(V + E)$	$\Theta(V^2)$
Best for	Sparse graphs	Dense graphs
Add vertex	$O(1)$ amortized	$\Theta(V^2)$ (resize)

Dimension	Adjacency List	Adjacency Matrix
Remove edge	$O(\text{degree})$	$O(1)$
Weighted graphs	Store weight with neighbor	Store weight in cell
Simplicity	More complex	Simpler for small graphs
Unweighted optimization	—	1 bit per entry

Dimension	BFS	DFS
Strategy	Explore by waves (breadth-first)	Explore deep, then backtrack
Data structure	FIFO queue	Stack (recursion)
Source restriction	Single source (typically)	Multiple sources (searches from every undiscovered vertex)
Distance computed	Shortest-path distance (edges)	No distances
Paths	Shortest paths (unweighted)	Any path
Edge classification	No back/forward edges in undirected graphs; no forward in directed	Tree, back, forward, cross
Typical uses	Shortest paths in unweighted graphs, bipartite checking	SCCs, topological sort, cycle detection, articulation points
Space (queue/stack)	Can be large ($O(V)$)	$O(V)$ recursion stack
Complexity	$O(V + E)$	$\Theta(V + E)$

Formulas & Equations

Shortest-path distance inequality (Lemma 20.1)

$\delta(s, v) \leq \delta(s, u) + 1$ for any edge (u, v)

- $\delta(s, v)$ = shortest-path distance (min edges) from s to v
- u, v = any adjacent vertices
- **Intuition:** A shortest path to v cannot be longer than a shortest path to u plus one more edge.

Discovery/finish time ordering (Corollary 20.8)

$u.d < v.d < v.f < u.f$ iff v is a proper descendant of u

- $u.d, u.f$ = discovery/finish times of vertex u
- **Parenthesis theorem:** For any two vertices, either $[u.d, u.f]$ and $[v.d, v.f]$ are disjoint, or one is nested in the other.

White-path theorem (Theorem 20.9)

v is a descendant of $u \Leftrightarrow$ at time $u.d$, there exists a path from u to v consisting entirely of white vertices.

BFS distance bounds

- $v.d \geq \delta(s, v)$ always (Lemma 20.2)
- After BFS terminates: $v.d = \delta(s, v)$ for all $v \in V$ (Theorem 20.5)

DFS timestamp invariants

- $1 \leq u.d < u.f \leq 2|V|$ for all $u \in V$
- u is WHITE before time $u.d$, GRAY between $u.d$ and $u.f$, BLACK after $u.f$

Queue monotonicity (Lemma 20.3)

If $Q = \langle v_1, v_2, \dots, v_r \rangle$ then $v_r.d \leq v_1.d + 1$ and $v_i.d \leq v_{i+1}.d$ for $i = 1, \dots, r-1$.

Corollary 20.4: d values of enqueued vertices are monotonically nondecreasing.

Edge classification by timestamps (Exercise 20.3-5)

Edge type Ordering

Tree/forward $u.d < v.d < v.f < u.f$

Back $v.d \leq u.d < u.f \leq v.f$

Cross $v.d < v.f < u.d < u.f$

Euler tour condition (Problem 20-3)

G has an Euler tour $\Leftrightarrow$ in-degree(v) = out-degree(v) for every vertex v (for strongly connected directed graph).

Rules, Laws & Theorems

Lemma 20.1 (Shortest-path inequality)

- **Statement:** For any edge $(u, v) \in E$, $\delta(s, v) \leq \delta(s, u) + 1$.
- **Proof:** If u reachable from s , then the shortest path to v cannot be longer than shortest path to u plus edge (u, v) . If u unreachable, $\delta(s, u) = \infty$.

Lemma 20.2 (BFS distance lower bound)

- **Statement:** $v.d \geq \delta(s, v)$ for all v at all times during BFS.
- **Proof:** By induction on number of ENQUEUE operations. Base: $s.d = 0 = \delta(s, s)$. Step: $v.d = u.d + 1 \geq \delta(s, u) + 1 \geq \delta(s, v)$.

Lemma 20.3 (Queue ordering invariant)

- **Statement:** During BFS, if $Q = \langle v_1, v_2, \dots, v_r \rangle$, then $v_r.d \leq v_1.d + 1$ and values are nondecreasing.
- **Proof:** By induction on queue operations. Dequeueing preserves property; enqueueing sets new vertex's $d = u.d + 1$.

Theorem 20.5 (Correctness of BFS)

- **Statement:** BFS discovers every vertex reachable from s , and $v.d = \delta(s, v)$ for all v . For any $v \neq s$ reachable from s , a shortest path from s to v is a shortest path from s to $v.\pi$ followed by $(v.\pi, v)$.
- **Proof by contradiction:** Let v be the vertex with minimal $\delta(s, v)$ having $v.d \neq \delta(s, v)$. Examine $u = \text{predecessor on some shortest path}$. When u dequeued, v is white/gray/black — each case contradicts $v.d > \delta(s, v)$.

Lemma 20.6 (BFS predecessor subgraph)

- **Statement:** The predecessor subgraph G_π produced by BFS is a breadth-first tree (connected, $|E_\pi| = |V_\pi| - 1$, unique shortest path from s to each vertex).

Theorem 20.7 (Parenthesis theorem)

- **Statement:** For any two vertices u, v in DFS, exactly one holds: (1) intervals $[u.d, u.f]$ and $[v.d, v.f]$ are disjoint, neither is descendant of other; (2) $[u.d, u.f]$ contains $[v.d, v.f]$ and u is ancestor of v ; (3) $[v.d, v.f]$ contains $[u.d, u.f]$ and v is ancestor of u .
- **Proof:** If $u.d < v.d$, either v is discovered while u is gray (v becomes descendant $\rightarrow$ nested) or after u finishes (disjoint). Symmetric case for $v.d < u.d$.

Corollary 20.8 (Nesting of descendants' intervals)

- **Statement:** v is a proper descendant of $u \Leftrightarrow u.d < v.d < v.f < u.f$.

Theorem 20.9 (White-path theorem)

- **Statement:** v is a descendant of u in DFS forest $\Leftrightarrow$ at time $u.d$, there is a path from u to v consisting entirely of white vertices.
- **Proof:** $(\Rightarrow)$ trivial. $(\Leftarrow)$ if path of white exists but v not descendant, let w be predecessor of v on path; w is descendant of u , so $w.f \leq u.f$. Then $u.d < v.d < w.f \leq u.f \rightarrow$ by Theorem 20.7, v is descendant of u — contradiction.

Theorem 20.10 (Undirected DFS edge types)

- **Statement:** In DFS of an undirected graph, every edge is either a tree edge or a back edge. No forward or cross edges.

Lemma 20.11 (Back edges $\Leftrightarrow$ cycles)

- **Statement:** Directed graph G is acyclic iff DFS yields no back edges.
- **Proof:** ($\Rightarrow$) Back edge (u, v) with v ancestor of u creates a cycle. ($\Leftarrow$) If G has a cycle, let v be first vertex discovered in the cycle, (u, v) the preceding edge; at $v.d$, all vertices in cycle are white $\rightarrow$ by white-path theorem u becomes descendant of $v \rightarrow (u, v)$ is a back edge.

Theorem 20.12 (Correctness of TOPOLOGICAL-SORT)

- **Statement:** TOPOLOGICAL-SORT produces a valid topological sort of a DAG.
- **Proof:** For any edge (u, v) in a DAG, v cannot be gray when edge explored (would be a back edge $\rightarrow$ cycle). If v is white, it becomes descendant of u , so $v.f < u.f$. If v is black, $v.f$ already set, and since u is still exploring, $u.f > v.f$. So $v.f < u.f$ for all edges $\rightarrow$ decreasing finish time order is valid.

Lemma 20.13 (SCC path property)

- **Statement:** If C and C' are distinct SCCs and there is a path from $u \in C$ to $u' \in C'$, then there cannot be a path from $v' \in C'$ back to $v \in C$.
- **Proof:** If both paths existed, all vertices would be mutually reachable $\rightarrow C$ and C' would be the same SCC.

Lemma 20.14 (SCC finish times)

- **Statement:** If there is an edge from C' to C ($u \in C', v \in C$) in G , then $f(C') > f(C)$ in the first DFS.
- **Proof:** Two cases: (1) $d(C') < d(C)$: first vertex x in C' discovered before any in C . By white-path theorem, all vertices in both C and C' become descendants of x , so $f(C') = x.f > f(C)$. (2) $d(C') > d(C)$: first vertex y discovered in C . Since no edge from C to C' (Lemma 20.13), all C' vertices are still white when y finishes, so $f(C') > f(C)$.

Corollary 20.15 (SCC transpose edges)

- **Statement:** If $f(C) > f(C')$, then G^T contains no edge from C to C' .
- **Proof:** Contrapositive of Lemma 20.14: if $f(C') < f(C)$, no edge from C' to C in $G \rightarrow$ no edge from C to C' in G^T .

Theorem 20.16 (Correctness of SCC algorithm)

- **Statement:** STRONGLY-CONNECTED-COMPONENTS correctly computes SCCs of G .
- **Proof by induction** on number of DFS trees in G^T . Basis ($k=0$): trivial. Step: root u in component C . $u.f = f(C)$ is maximum among unvisited components. By Corollary 20.15, G^T has no edges from C to other unvisited components; by white-path theorem, all C vertices are

descendants of u ; never visits outside C ; thus the tree = exactly one SCC.

Data Structures & Types

- **Graph $G = (V, E)$:** Vertex set $G.V$, edge set $G.E$.
- **Adjacency list:** Array $G.Adj$ of $|V|$ linked lists (or hash tables). For weighted graphs, store $(v, w(u,v))$ pairs.
- **Adjacency matrix:** $|V| \times |V|$ matrix. For unweighted: 1 bit per entry. For weighted: store weight or NIL/ ∞ /0.
- **FIFO Queue:** Used in BFS to maintain frontier of gray vertices. Contains vertices from at most 2 consecutive distance levels (Lemma 20.3).
- **Linked list:** Used in TOPOLOGICAL-SORT to collect vertices in order; insertion at front is $O(1)$.
- **Recursion stack (implicit):** Used in DFS-VISIT to track path; depth $\leq |V|$.
- **Vertex attributes:** color (WHITE/GRAY/BLACK), d (distance/discovery time), f (finish time), π (predecessor), cc (connected component label).
- **Hash table:** Can replace adjacency lists for $O(1)$ expected edge lookup (Exercise 20.1-8).
- **Component graph G_SCC :** Each SCC contracted to one vertex; always a DAG.

Edge Cases & Common Pitfalls

- **Unreachable vertices in BFS:** Stay WHITE, $d = \infty$, $\pi = \text{NIL}$, never enqueued. Not in V_π .
- **BFS graph with isolated vertices:** DFS and BFS handle differently. BFS from s leaves isolated vertices undiscovered. DFS loops over all vertices, discovers every vertex.
- **Adjacency-list size for undirected graphs:** Each edge (u, v) appears in both $Adj[u]$ and $Adj[v]$, total length $= 2|E|$.
- **BFS tree depends on adjacency list order:** Distances d are invariant (Exercise 20.2-5) but the breadth-first tree can differ.
- **Single-bit color suffices for BFS/DFS:** Removing the BLACK-assigning line still produces correct distances (Exercises 20.2-3, 20.3-4). Colors can be eliminated entirely using distance/ D -value checks.
- **DFS gives different forests on same graph:** Order of vertices in main loop and order in adjacency lists affect discover/finish times and forest structure.
- **DFS-VISIT recursion depth:** Can be $O(V)$ in worst case (e.g., a path graph). Could overflow recursion stack; can rewrite with explicit stack (Exercise 20.3-6).
- **Self-loops:** Only possible in directed graphs. Classified as back edges in DFS.
- **BFS on adjacency matrix:** $O(V^2)$ rather than $O(V + E)$ because scanning each row takes $O(V)$ even for isolated vertices (Exercise 20.2-4).

- **Topological sort requires DAG:** Running TOPOLOGICAL-SORT on a graph with cycles produces an invalid ordering. The algorithm doesn't detect cycles — need to check for back edges separately.
- **Kahn's algorithm cycle detection:** If algorithm finishes without outputting all vertices, the remaining vertices form cycles.
- **SCC algorithm requires two DFS passes:** Using original graph instead of transpose in the second pass with increasing finish times does NOT always work (Exercise 20.5-3).
- **Adding an edge to a graph:** Can only decrease the number of SCCs (merging components), never increase them (Exercise 20.5-1).
- **Multiple edges between vertices:** In adjacency-list representation, a multigraph stores duplicate entries. Exercise 20.1-4 shows how to remove duplicates.
- **Universal sink detection:** Using adjacency matrix, can find in $O(V)$ time by starting at (1,1), moving right on 1 and down on 0 (Exercise 20.1-6).
- **Single-bit color suffices for BFS:** If line 18 ($u.color = BLACK$) is removed, BFS still produces correct distances because GRAY vs BLACK distinction is not needed for correctness — only WHITE vs non-WHITE matters for discovery. BLACK only helps conceptual understanding. For DFS, same holds (Exercise 20.3-4).
- **Semiconnected vs strongly connected:** Semiconnected requires that for all (u,v) , at least one direction is reachable. This is weaker than strong connectivity (both directions required).

Case Studies & Examples

BFS on directed graph (Figure 20.2, Exercise 20.2-1)

- **What:** BFS on a directed graph with 6 vertices (1, 2, 3, 4, 5, 6) using vertex 3 as source.
- **Method:** Run $BFS(G, 3)$ on the graph shown in Figure 20.2(a). Adjacency list order is unspecified but distances are invariant.
- **Results:** Distances d from vertex 3: $3.d=0$; neighbors reachable from 3 get $d=1$; their neighbors get $d=2$, etc. Vertices not reachable from 3 have $d=\infty$.
- **Significance:** Demonstrates that BFS on directed graphs only reaches vertices along directed paths from the source.

BFS on undirected graph (Figure 20.3)

- **What:** BFS on an undirected graph showing queue evolution with two consecutive distance levels in the queue.
- **Method:** Each part of Figure 20.3 shows the graph state at the start of each while-loop iteration. Tan = queue (frontier), light blue = behind frontier (dequeued), orange = currently dequeued vertex.
- **Results:** Queue always contains vertices with d values like $\langle k, k, k, k+1, k+1 \rangle$.
- **Significance:** Illustrates Lemma 20.3 — the queue holds at most two distinct distance levels.

DFS on directed graph (Figure 20.4)

- **What:** DFS on a directed graph with vertices labeled by letters, showing discovery/finish timestamps and edge classification.
- **Method:** DFS with alphabetical vertex ordering and alphabetical adjacency lists.
- **Results:** Each vertex labeled with d/f timestamps. Edge types labeled T (tree), B (back), F (forward), C (cross).
- **Significance:** Demonstrates all four edge types and parenthesis structure of timestamps.

Professor Bumstead's clothing DAG (Figure 20.7)

- **What:** Topological sort of a DAG representing clothing dependencies (socks before shoes, etc.).
- **Method:** Each vertex is an item of clothing; edge (u, v) means u goes on before v. Run DFS, order vertices by decreasing finish time.
- **Results:** A valid dressing order: undershorts, pants, belt, shirt, tie, jacket, socks, shoes, watch. All directed edges go left-to-right.
- **Significance:** Classic motivating example for topological sort. Shows real-world precedence constraints.

SCC decomposition (Figure 20.9)

- **What:** Computing SCCs of a directed graph with vertices {a, b, c, d, e, f, g, h}.
- **Method:** (1) First DFS on G to compute finish times. (2) Compute G^T . (3) Second DFS on G^T processing vertices in decreasing finish-time order from step 1.
- **Results:** Components: {a, b, e}, {c, d}, {f, g}, {h}. The component graph is a DAG.
- **Significance:** Demonstrates that each DFS tree in the second pass corresponds exactly to one SCC. Orange vertices (b, c, g, h) are roots of G^T 's DFS trees.

Lecture hall activity selection (topological sort motivation, Figure 20.8)

- **What:** DAG with vertices {m, n, o, p, q, r, s, t, u, v, w, x, y, z} used for topological sort exercise.
- **Results:** Exercise 20.4-1 asks to compute topological order with alphabetical processing.
- **Significance:** Exercise 20.4-2 extends to counting number of paths between two vertices (4 paths from p to v).

Diagrams & Visuals

Figure 20.2: Directed graph representations

(a) Directed graph G (6 vertices, 8 edges):

Vertices: 1, 2, 3, 4, 5, 6

Edges: 1→2, 1→4, 2→5, 3→5, 3→6, 4→2, 5→4, 6→6 (self-loop)

(b) Adjacency-list representation:

1: → 2 → 4
2: → 5
3: → 5 → 6
4: → 2
5: → 4
6: → 6

(c) Adjacency matrix (6×6):

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

BFS queue evolution (Figure 20.3 pattern):

Initial: Q = [s] (d=0)
Wave 1: Q = [a, b] (d=1, d=1)
Wave 2: Q = [b, c, d] (d=1, d=2, d=2)
Wave 3: Q = [c, d, e, f] (d=2, d=2, d=3, d=3)

Note: At any time, Q contains vertices with at most two distinct d values (Lemma 20.3)

Legend: WHITE ○ → GRAY ● → BLACK ●

Frontier (tan) = GRAY vertices in queue

Behind frontier (light blue) = BLACK vertices (dequeued)

DFS parenthesis structure (Figure 20.5):

Intervals as nested parentheses:

a (b (d (e ()) f ()) c ())

Parenthesis theorem visualization:

a: [1, 8]	┌───────────┐	
b: [2, 7]	└──────────┘	
c: [8, 9]		└───┘
d: [3, 6]	└────────┘	
e: [4, 5]	└───┘	
f: [9, 10]		└───┘
	1 2 3 4 5 6 7 8 9 10	

Legend:

┌───┐ = interval between discovery and finish

Nesting = ancestor relationship

Disjoint = no ancestor relationship

DFS forest with edge types:

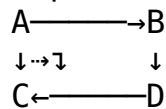
Tree edges (→): bold, part of DFS forest

Back edges (↗): point to ancestor (including self-loops)

Forward edges (↘): to descendant, not part of tree

Cross edges (- - →): between branches or different trees

Example:

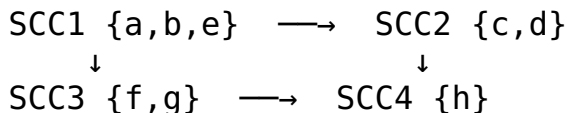


Tree: $A \rightarrow B$, $A \rightarrow C$ (from first DFS tree)

Back: $C \rightarrow A$, $B \rightarrow B$ (self-loop)

Cross: $D \rightarrow C$ (same tree, no ancestor relation)

Component graph G_SCC (Figure 20.9c):



This is always a DAG.

The second DFS on G^T visits SCCs in reverse topological order:

$SCC4 \rightarrow SCC3 \rightarrow SCC2 \rightarrow SCC1$

Kahn's algorithm visualization:

Step 1: Compute in-degree for each vertex

Step 2: Queue = all in-degree-0 vertices

Step 3: Dequeue, output, decrement neighbors' in-degrees

Step 4: Repeat

Example (clothing DAG):

undershorts(0)* $\rightarrow$ pants(1) $\rightarrow$ belt(2) $\rightarrow$ jacket(2)

socks(0)* $\rightarrow$ shoes(2)

shirt(0)* $\rightarrow$ belt(2), tie(1) $\rightarrow$ jacket(2)

watch(0)*

* = initial in-degree 0, enqueued first

Process in queue order, decrementing as we go.

End-of-Chapter Material

Key Terms (from throughout the chapter): - adjacency-list representation, adjacency-matrix representation, weighted graph, sparse/dense graph, transpose, square of a graph, incidence matrix, universal sink - breadth-first search (BFS), source vertex, shortest-path distance $\delta(s, v)$, breadth-first tree, predecessor subgraph, tree edge - depth-first search (DFS), depth-first forest, discovery time, finish time, parenthesis structure, white-path theorem - tree edge, back edge, forward edge, cross edge - topological sort, directed acyclic graph (DAG) - strongly connected component (SCC), component graph - articulation point, bridge, biconnected component, Euler tour, semiconnected, singly connected, planar graph

Exercises with Solutions:

20.1-1: Out-degree: $O(V + E)$ by scanning all adjacency lists and counting. In-degree: $O(V + E)$ by initializing an in-degree array to 0, then for each vertex u , for each v in $\text{Adj}[u]$, increment $\text{in-degree}(v)$.

20.1-2: Complete binary tree on 7 vertices (heap numbering 1-7): - Adjacency list: 1: 2,3; 2: 1,4,5; 3: 1,6,7; 4: 2; 5: 2; 6: 3; 7: 3 - Adjacency matrix: 7×7 with 1s at symmetric positions for edges

20.1-3: Transpose from adjacency list: create new list of size $|V|$. For each u , for each v in $\text{Adj}[u]$, append u to new $\text{Adj}_T[v]$. $O(V + E)$. From matrix: set $A_T[i][j] = A[j][i]$. $O(V^2)$.

20.1-4: For each vertex u , create a boolean array or use sorting on $\text{Adj}[u]$ to detect duplicates; skip self-loops when $v = u$. $O(V + E)$ with appropriate data structures.

20.1-5: G^2 from adjacency list: for each u , for each v in $\text{Adj}[u]$, for each w in $\text{Adj}[v]$, add w to $\text{Adj}_2[u]$ (if $w \neq u$). $O(V \cdot E \cdot \text{degree})$ worst-case. From matrix: compute A^2 using boolean OR/multiplication. $O(V^3)$ naive, $O(V^{\omega})$ with Strassen.

20.1-6: Universal sink in $O(V)$: Start at $(1,1)$. If $A[i][j] = 1$, move right ($i++$); if 0, move down ($j++$). At end, verify candidate row has all 0s and column has all 1s except diagonal.

20.1-7: BB^T entry (i,j) = number of edges from i to j minus self-loops. B is $|V| \times |E|$ incidence matrix.

20.1-8: Hash table per adjacency list: expected $O(1)$ edge lookup. Disadvantage: hash table overhead, poor cache performance, no easy iteration. Alternatives: a balanced BST ($O(\log \text{degree})$ lookup), or storing edges in a sorted array ($O(\log \text{degree})$ with binary search).

20.2-1: BFS from vertex 3 on graph of Figure 20.2(a): 3.d=0, 5.d=1, 6.d=1, 4.d=2 (via 5), 2.d=2 (via 5 or via 4), 1.d=3 (via $4 \rightarrow 2$ or $2 \rightarrow 1$ if edge exists, else 1 is unreachable). π values reflect tree edges.

20.2-2: BFS from u on Figure 20.3 undirected graph. Results depend on adjacency list order. Distances are invariant, only tree structure may vary.

20.2-3: Single-bit color suffices: remove line 18 ($u.\text{color} = \text{BLACK}$). GRAY and BLACK are treated identically for discovery decisions (only WHITE matters). To eliminate colors entirely, use d values: a vertex is discovered if $d \neq \infty$.

20.2-4: BFS with adjacency matrix: $O(V^2)$ — scanning each row takes $O(V)$, and we must scan all rows because each vertex's adjacency matrix row has V entries.

20.2-5: Distances d are independent of adjacency list order because each vertex is discovered at the first time a neighbor is dequeued that has an edge to it, and this always yields the shortest distance. Tree may differ

because the parent of a vertex could be any of its already-discovered neighbors.

20.2-6: Directed graph: $V = \{s, a, b\}$, edges $s \rightarrow a$, $s \rightarrow b$, $a \rightarrow b$. The tree edges $\{(s,a), (a,b)\}$ produce shortest paths but cannot be produced by BFS because BFS will discover b from s (distance 1) not from a (distance 2).

20.2-7: Two-coloring problem (bipartite check). BFS from each vertex; color source FACE, then alternate (neighbors get opposite color). If a conflict (neighbor already colored with same color) $\rightarrow$ impossible. $O(n + r)$.

20.2-8: Tree diameter: pick arbitrary vertex v , BFS to find farthest vertex u . BFS from u to find farthest vertex w . Distance from u to w is diameter. $O(V)$.

20.3-1: 3×3 chart for directed graph:

From To WHITE GRAY BLACK

WHITE	—	Tree	Forward/Cross
GRAY	—	Back	—
BLACK	—	—	—

For undirected: only WHITE $\rightarrow$ GRAY (tree) and GRAY $\rightarrow$ WHITE (back) possible.

20.3-2: DFS on Figure 20.6 with alphabetical order. [Student should compute timestamps — the graph has vertices $\{s, t, u, v, w, x, y, z\}$.]

20.3-3: Parenthesis structure for Figure 20.4: shows nested intervals corresponding to each vertex's $[d, f]$ range.

20.3-4: Single-bit color for DFS: remove line 10 of DFS-VISIT ($u.color = BLACK$). BLACK doesn't affect edge classification because classification uses GRAY detection for back edges.

20.3-5: Edge classification by timestamps: - Tree/forward: $u.d < v.d < v.f < u.f$ - Back: $v.d \leq u.d < u.f \leq v.f$ - Cross: $v.d < v.f < u.d < u.f$

20.3-6: Rewrite DFS with explicit stack: push (vertex, iterator state) pairs onto stack.

20.3-7: Counterexample: $u \rightarrow w \rightarrow v$ and also $u \rightarrow v$. If DFS visits $u \rightarrow w$ first, then $w \rightarrow v$ (v discovered from w), then $u \rightarrow v$ is explored later — v already discovered, and $u.d < v.d$, but v is descendant of w not u .

20.3-8: Counterexample: chain $u \rightarrow w \rightarrow v$. DFS from u discovers u , then w (so w discovered, v undiscovered). If DFS goes $u \rightarrow w \rightarrow \dots$ long path, then eventually discovers v . $v.d$ could be $> u.f$ if search backtracks past u before reaching v .

20.3-9: Modify DFS-VISIT to print edge type: when exploring (u,v) , check $v.color$ — WHITE: tree; GRAY: back; BLACK: check timestamps for forward/cross.

20.3-10: A vertex u can be in a DFS tree containing only u if it is chosen as a new source in the main DFS loop and has no outgoing edges to undiscovered vertices (all neighbors already discovered/black), and no undiscovered vertices reach it.

20.3-11: Euler tour in undirected graph: DFS that follows each edge twice (once in each direction). Use pennies to mark traversed edges. $O(V + E)$.

20.3-12: Connected components: modify DFS-VISIT to pass a counter; assign $v.cc = \text{counter}$ each time DFS-VISIT discovers a vertex. Each call to DFS-VISIT from the main loop starts a new component.

20.4-1: Topological sort of Figure 20.8 dag with alphabetical order. [Student should compute finish times and then order by decreasing finish time.]

20.4-2: Count paths from a to b in DAG: compute topological order, then for each vertex in topological order, sum path counts from incoming neighbors. $\text{paths}[a] = 1$. $O(V + E)$. Example (Figure 20.8): 4 paths from p to v .

20.4-3: Cycle detection in undirected graph $O(V)$: BFS/DFS, if $|E| \geq |V|$ for a connected graph, there must be a cycle. More precisely, if the graph has more than $|V| - 1$ edges, it contains a cycle.

20.4-4: False. Vertex ordering that minimizes “bad” edges (edges going backward) is the minimum feedback arc set problem, which is NP-hard. Topological sort on a cyclic graph doesn’t minimize bad edges.

20.4-5: Kahn’s algorithm: compute in-degree array. Queue all in-degree-0 vertices. While queue not empty, dequeue vertex, output it, for each neighbor decrement in-degree; if in-degree becomes 0, enqueue. If not all vertices output $\rightarrow$ graph has a cycle. $O(V + E)$.

20.5-1: Adding an edge can only decrease the number of SCCs (by merging components) or keep it the same. Never increases the number of SCCs.

20.5-2: SCC on Figure 20.6: first DFS computes finish times; second DFS on G^T processes in decreasing finish time order; each tree in the forest is one SCC.

20.5-3: No — using original graph with increasing finish times does not always produce correct SCCs. Counterexample exists where components are merged incorrectly.

20.5-4: $((G^T)_{\text{SCC}})^T = G_{\text{SCC}}$. Proof: SCCs are the same in G and G^T . Reversing edges twice returns to original edges.

20.5-5: Compute SCCs, create an array $\text{comp}[1..k]$, iterate over all edges (u, v) , if $\text{comp}[u] \neq \text{comp}[v]$, add edge $(\text{comp}[u], \text{comp}[v])$ to component graph, using a set to avoid duplicates. $O(V + E)$.

20.5-6: Within each SCC, keep a cycle that touches all vertices (if possible, a Hamiltonian path or just a spanning tree). Between SCCs, keep at most one edge per direction. Minimum edges = $|V_{\text{SCC}}| + |E_{\text{SCC_essential}}|$.

20.5-7: Semiconnected check: compute SCCs, build component graph, topological sort, verify that there is an edge between each consecutive pair of SCCs in topological order. If any consecutive pair lacks an edge, graph is not semiconnected.

20.5-8: Compute $\min(u)$ using SCC decomposition + DP on DAG. Within each SCC, $\min(u)$ = minimum label in that component. On component graph, propagate minima in topological order.

Problems: - **20-1 (BFS edge classification):** Prove undirected BFS has no back/forward edges; cross edges satisfy $v.d = u.d$ or $v.d = u.d + 1$. Directed BFS has no forward edges; cross edges: $v.d \leq u.d + 1$; back edges: $0 \leq v.d \leq u.d$. - **20-2 (Articulation points, bridges, biconnected components):** Use low-link values ($v.low = \min(v.d, w.d$ for back edges (v,w) , $low(child)$). Root is articulation point iff ≥ 2 children. Nonroot is articulation point iff child s has $low(s) \geq v.d$. Bridge iff edge not on any simple cycle. - **20-3 (Euler tour):** Exists iff $\text{in-degree}(v) = \text{out-degree}(v)$ for all v (in strongly connected digraph). Find by merging edge-disjoint cycles. - **20-4 (Reachability):** Compute $\min(u)$ using DFS on G^T : start from smallest labeled vertex, propagate reachability. - **20-5 (Planar graph operations):** INSERT in $O(1)$ amortized using potential function. NEWEST-NEIGHBOR in $O(1)$ worst-case.

Cross-Chapter Links

- **Requires knowledge of:** Ch. 10 (stacks, queues, linked lists), Appendix B (graph definitions, trees), Section 16.1 (aggregate analysis for BFS/DFS running time analysis), Ch. 15 (greedy algorithms — referenced for MST connection)
- **Referenced in later chapters:** Ch. 21 (MST builds on BFS/DFS ideas), Ch. 22 (shortest paths — Dijkstra uses BFS-like ideas; Bellman-Ford uses graph search), Ch. 23 (all-pairs shortest paths — adjacency matrix representation assumed), Ch. 24 (maximum flow — uses graph search for augmenting paths), Ch. 25 (bipartite matching — extends SCC ideas), Ch. 34 (NP-completeness — uses graph concepts)
- **Other references:** Section 16.1 (aggregate analysis used in BFS/DFS running time proofs). The SCC algorithm is adapted from Aho, Hopcroft & Ullman; DFS edge classification referenced in Lemma 20.11.
- **Chapter notes reference:** Moore (BFS), Lee (BFS), Hopcroft & Tarjan (adjacency-list advocacy, DFS importance), Tarjan (SCC linear-time algorithm), Kosaraju & Sharir (SCC algorithm), Dijkstra (SCC algorithm), Gabow (SCC rediscovery), Knuth (topological sort).

Cross-Cutting Topics

Design Paradigms & Meta-Methods

- **Graph search (BFS/DFS):** Fundamental algorithmic pattern for exploring graphs. BFS uses a queue (FIFO); DFS uses a stack (LIFO/recursion). Running time $O(V + E)$ for both when using adjacency lists.

The choice between them depends on the problem: shortest paths → BFS; reachability, cycle detection, connectivity → either; topological sort, SCCs → DFS. (Ch. 20)

- **Predecessor subgraph / parent pointers:** Common technique to reconstruct paths after graph search. Set $v.\pi = u$ when edge (u,v) first discovers v . The resulting parent pointers form a tree/forest that encodes discovery relationships. (Ch. 20)
- **Color marking:** WHITE/GRAY/BLACK (or VISITED/IN-PROGRESS/DONE) is a general pattern for tracking state during graph traversal. WHITE = undiscovered, GRAY = in progress (on recursion stack / in queue), BLACK = finished. (Ch. 20)
- **Timestamping:** Each vertex gets discovery time and finish time during DFS. Used to reason about ancestor relationships, parenthesis structure, and to prove properties. (Ch. 20)
- **Graph transpose:** G^T reverses all edges. Used in SCC algorithm. The transpose preserves SCC structure but reverses direction of component graph edges. (Ch. 20)
- **Component graph contraction:** Contract SCCs into single vertices to yield a DAG, enabling divide-and-conquer on directed graphs. (Ch. 20, referenced in Ch. 22, 24, 25)
- **Two-pass algorithm:** The SCC algorithm uses two DFS passes (first on G , second on G^T with specific vertex ordering). This pattern of running the same algorithm with different ordering on a transformed input is reusable. (Ch. 20)
- **Divide-and-conquer:** Graph search decomposes the graph by exploring components one at a time. SCC decomposition further partitions a directed graph into DAG-structured components. (Ch. 4, Ch. 20)

Proof & Argument Patterns

- **Proof by induction on queue operations** (Lemma 20.3): Used to prove queue invariants in BFS. Inductive hypothesis about queue contents after each operation. (Ch. 20)
- **Proof by contradiction with minimal counterexample** (Theorem 20.5): Among all vertices with incorrect d value, pick one with minimal $\delta(s, v)$. Show contradiction by examining cases when its predecessor u is dequeued. Common pattern for correctness proofs of search/optimization algorithms. (Ch. 20)
- **Proof by parentheses/nesting:** Theorem 20.7 and Corollary 20.8 use the mathematical structure of nested intervals to characterize ancestor relationships. This is a general technique: map program events to intervals and use interval containment to reason about relationships. (Ch. 20)
- **White-path theorem** (Theorem 20.9): If-then reasoning about reachability and discovery order. Forward direction is straightforward; reverse uses proof by contradiction with the nearest non-descendant vertex. (Ch. 20)
- **Contrapositive** (Corollary 20.15): Used to derive the key property of SCC algorithm — reversing Lemma 20.14 and applying to G^T proves correctness of the second DFS pass. (Ch. 20)

- **Induction on number of trees:** Theorem 20.16 uses induction on the number of DFS trees in the second pass to prove that each tree corresponds to one SCC. (Ch. 20)
- **Adversarial argument** (Exercise 20.2-6): To show BFS cannot produce certain edge sets, construct a graph where the desired set cannot arise from BFS behavior regardless of adjacency list ordering. (Ch. 20)
- **Aggregate analysis:** Running time of BFS/DFS is derived by counting total operations across all vertices, not per-vertex. Each vertex dequeued once, each adjacency list scanned once $\rightarrow O(V + E)$. (Ch. 16, Ch. 20)

Probability & Statistics Foundation

- **Expected edge lookup with hash tables** (Exercise 20.1-8): Under uniform independent hashing, expected time to check if edge (u, v) exists is $O(1)$, based on the length of chain at bucket $h(v)$ in $\text{Adj}[u]$'s hash table. Analysis uses the uniform hashing assumption. (Ch. 20)

Mnemonics & Memory Aids

- **BFS queue invariant:** "Q has at most two consecutive d-values" — remember $Q = [k, k, \dots, k, k+1, k+1, \dots, k+1]$
- **DFS edge colors $\rightarrow$ types:** "White = Tree, Gray = Back, Black = Forward/Cross" (W-T, G-B, B-F/C)
- **Edge classification by timestamps:** "Tree/Forward: d-v-u; Back: u inside v; Cross: completely disjoint" — visualize the interval nesting
- **SCC algorithm:** "First pass: finish times; Second pass: transposed graph, decreasing finish time; Each tree = one SCC"
- **Topological sort:** "Finish times descending = topological ordering"
- **Kosaraju's algorithm steps:** "DFS, Transpose, DFS(dec finish), Output trees"
- **Articulation points:** "Root: 2+ children; Nonroot: child can't climb over me" ($v.\text{low}(\text{child}) \geq v.d$)
- **Theorem 20.10:** "Undirected DFS has no cross or forward edges" — UC (Undirected $\rightarrow$ Cross/Forward = $\emptyset$)
- **BFS vs DFS queue/stack:** "BFS = Bowl (Queue, First-In-First-Out), DFS = Dish (Stack, Last-In-First-Out)"
- **4 edge types:** "Tree (family), Back (ancestor), Forward (descendant not tree), Cross (other)" — T, B, F, C

People & Dates

- **Edward F. Moore** (1959): Discovered BFS in context of finding paths through mazes.
- **C. Y. Lee** (1961): Independently discovered BFS for routing wires on circuit boards.
- **John Hopcroft & Robert Tarjan** (1970s): Advocated adjacency-list representation over adjacency-matrix for sparse graphs; first to recognize algorithmic importance of DFS.
- **Robert Tarjan** (1972): Gave first linear-time algorithm for finding strongly connected components.

- **S. R. Kosaraju** (unpublished): Co-developed the SCC algorithm presented in the chapter (the two-pass approach on G and G^T).
- **M. Sharir** (1981): Also independently developed the same SCC algorithm.
- **Edsger Dijkstra** (1976): Developed an SCC algorithm based on contracting cycles (Chapter 25 of his book).
- **Harold Gabow**: Rediscovered Dijkstra's cycle-contracting SCC algorithm.
- **Donald Knuth**: First to give a linear-time algorithm for topological sorting.
- **Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein**: Authors of CLRS (4th edition, 2022).

Ethics & Professional Practice

- **Algorithmic fairness**: The book mentions that algorithms suggest sentences for convicted criminals (Preface) — an ethical concern about algorithmic decision-making in high-stakes contexts. Graph algorithms like those in this chapter could be components of such systems, and understanding their limitations is crucial.
- **Algorithmic transparency**: The authors emphasize understanding what algorithms do, how they operate, and what their limitations are — this is an ethical imperative for computer scientists and citizens alike.
- **Gender-neutral language**: The 4th edition explicitly revised language to be more inclusive (e.g., “traveling-salesperson” instead of “traveling-salesman”), reflecting the importance of making CS welcoming to everyone. (Ch. 20 follows this convention.)
- **Dual-use concerns**: Graph algorithms (BFS, DFS, SCC decomposition, topological sort) are used in applications from routing to social network analysis; the same techniques power recommendation systems and surveillance. Understanding the algorithms includes understanding potential for misuse.